

TECH

LIVE-CODING

JAVASCRIPT / ALGORITHMS

TYPESCRIPT

REACT

JAVASCRIPT / ALGORITHMS

30 ЗАДАЧ

7 ДНЕЙ

КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК

leetcode / mdn / classic patterns

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	DEBOUNCE E CURRY	THROTTLE	ONCE	MEMOIZ	DEBOUNCE E CURRY	THROTTLE	ONCE	MEMOIZ
	Q1 · Q2 · Q3 · Q4 · Q5							
ДЕНЬ 2 СРЕДНИЙ	DEEP CLONE FLATTEN	DEEP EQUAL	FLAT	UN	DEEP CLONE FLATTEN	DEEP EQUAL	FLAT	UN
	Q6 · Q7 · Q8 · Q9							
ДЕНЬ 3 ТЯЖЁЛЫЙ	PROMISE.ALL T SEQUENTIAL	POOL	RETRY	TIMEOU	PROMISE.ALL T SEQUENTIAL	POOL	RETRY	TIMEOU
	Q10 · Q11 · Q12 · Q13 · Q14							
ДЕНЬ 4 СРЕДНИЙ	CHUNK DIFF	GROUP	UNIQ	INTERSECT	CHUNK DIFF	GROUP	UNIQ	INTERSECT
	Q15 · Q16 · Q17 · Q18 · Q19							
ДЕНЬ 5 ТЯЖЁЛЫЙ	EVENT EMITTER R GENERATOR	OBSERVABLE	ITERATO		EVENT EMITTER R GENERATOR	OBSERVABLE	ITERATO	
	Q20 · Q21 · Q22 · Q23							
ДЕНЬ 6 ТЯЖЁЛЫЙ	FETCH POLLING U	STREAM	CACHE	LR	FETCH POLLING U	STREAM	CACHE	LR
	Q24 · Q25 · Q26 · Q27							
ДЕНЬ 7 СРЕДНИЙ	SCHEDULER	PRIORITY QUEUE	SCHEDULE		SCHEDULER	PRIORITY QUEUE	SCHEDULE	
	Q28 · Q29 · Q30							

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1

НАГРУЗКА

СРЕДНИЙ

5 ВОПРОСОВ

DEBOUNCE**THROTTLE****ONCE****MEMOIZE****CURRY**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DEBOUNCE

THROTTLE

ONCE

MEMOIZE

CURRY

ВОПРОСЫ ДНЯ

Q01 Реализуй `debounce(fn, delay)`Q02 Реализуй `throttle(fn, interval)`Q03 Реализуй `once(fn)`Q04 Реализуй `memoize(fn)`Q05 Реализуй `curry(fn)`

Реализуй `debounce(fn, delay)`

ОПИСАНИЕ

Возвращает обёртку которая откладывает вызов `fn` на `delay ms`. Каждый новый вызов сбрасывает таймер. Полезно для `search input`, `resize`, `scroll`.

КАК РАБОТАЕТ

- 01 `clearTimeout` при каждом вызове.
- 02 `setTimeout` для запланированного вызова.
- 03 Сохраняй `context (this)` и `args`.
- 04 Опционально: `leading / trailing edge`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Search input`.
- 02 `Resize handlers`.
- 03 `Form validation`.

ПОДВОДНЫЕ КАМНИ

- 01 Не сохранять `this` – поломка контекста.
- 02 Без `trailing` – последний вызов теряется.

МОГУТ ДОСПРОСИТЬ

- 01 Чем `throttle` отличается?
- 02 Как сделать `leading-debounce`?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#debounce>

ВОПРОС Q02

Реализуй throttle(fn, interval)

ОПИСАНИЕ

Возвращает обёртку которая вызывает fn не чаще раза в interval ms. Полезно для scroll, mousemove, drag. Edge case: leading и trailing вызовы.

КАК РАБОТАЕТ

- 01 Запоминай lastCall timestamp.
- 02 Если now - lastCall >= interval – вызывай.
- 03 Иначе планируй setTimeout на остаток.
- 04 Trailing: последний вызов после паузы.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Scroll handlers.
- 02 Mousemove tracking.
- 03 Rate limiting UI.

ПОДВОДНЫЕ КАМНИ

- 01 Без trailing – последний event потерян.
- 02 Без leading – задержка на первый вызов.

МОГУТ ДОСПРОСИТЬ

- 01 Чем debounce от throttle отличается?
- 02 Как тестировать throttle?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#throttle>

Реализуй `once(fn)`

ОПИСАНИЕ

Обёртка которая вызывает `fn` только при первом вызове, далее возвращает результат первого. Полезно для `init`, `lazy initialization`, `idempotent calls`.

КАК РАБОТАЕТ

- 01 Сохраняй `called` флаг + `result`.
- 02 `return result` для последующих вызовов.
- 03 Передай `context` + `args` в `fn`.
- 04 Опционально: освобождение ссылки на `fn`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Init code`.
- 02 `Lazy initialization`.
- 03 `Idempotent callbacks`.

ПОДВОДНЫЕ КАМНИ

- 01 Заккрытие держит `fn` даже после первого вызова.
- 02 `Race condition` — два `sync` вызова до завершения.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое идемпотентность?
- 02 Как реализовать `once` для `async`?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#once>

ВОПРОС Q04

Реализуй memoize(fn)

ОПИСАНИЕ

Кэширует результат fn по аргументам. Если уже считали – return из cache. Полезно для pure-functions с дорогим вычислением (fibonacci, parsing).

КАК РАБОТАЕТ

- 01 Map / Object cache.
- 02 Key = JSON.stringify(args) или resolver.
- 03 Если ключ есть – return cached.
- 04 Иначе – вычисли и сохрани.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Pure functions.
- 02 Expensive computations.
- 03 Recursive (fib, factorial).

ПОДВОДНЫЕ КАМНИ

- 01 Cache растёт без bound – memory leak.
- 02 JSON.stringify для objects – медленно.
- 03 Не memoize side effect.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое WeakMap-cache?
- 02 Как ограничить размер кэша?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#memoize>

Реализуй `curry(fn)`

ОПИСАНИЕ

Преобразует `fn(a,b,c)` в `curry(fn)(a)(b)(c)` или `curry(fn)(a,b)(c)`. Полезно для composition, partial application, point-free style.

КАК РАБОТАЕТ

- 01 Если `args.length >= fn.length` – вызывай.
- 02 Иначе – return функцию которая собирает доп. args.
- 03 Замыкание над собранными args.
- 04 Поддержка partial: `curry(fn)(a,b)(c)`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 FP composition.
- 02 Partial application.
- 03 Point-free style.

ПОДВОДНЫЕ КАМНИ

- 01 Не учитывать `fn.length` с rest-params.
- 02 Mutate accumulator вместо нового array.

МОГУТ ДОСПРОСИТЬ

- 01 Чем partial application отличается?
- 02 Что такое point-free style?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#curry>

ДЕНЬ 2

НАГРУЗКА

СРЕДНИЙ

4 ВОПРОСА

DEEP CLONE**DEEP EQUAL****FLAT****UNFLATTEN**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DEEP CLONE DEEP EQUAL FLAT UNFLATTEN

ВОПРОСЫ ДНЯ

Q06 Реализуй `deepClone(obj)`Q07 Реализуй `deepEqual(a, b)`Q08 Реализуй `flat(arr, depth = 1)`Q09 Реализуй `unflatten({a.b.c: 1})`

Реализуй `deepClone(obj)`

ОПИСАНИЕ

Глубокая копия объекта. Все nested objects/arrays – независимые копии.
Структурированный clone: `structuredClone` (modern) или ручная рекурсия + `circular check`.

КАК РАБОТАЕТ

- 01 `structuredClone(obj)` – modern API.
- 02 Ручная: `typeof` check, `Array` vs `Object`.
- 03 `WeakMap` для `circular refs`.
- 04 Спеc types: `Date`, `Map`, `Set`, `RegExp`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 State snapshots.
- 02 Immutable updates.
- 03 Test fixtures.

ПОДВОДНЫЕ КАМНИ

- 01 `JSON.parse(JSON.stringify(x))` – теряет `undefined`, `Date`, `Map`, `functions`.
- 02 Без `circular check` – `stack overflow`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `structured clone algorithm`?
- 02 Чем `shallow` от `deep clone` отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/structuredClone>

ВОПРОС Q07

Реализуй `deepEqual(a, b)`**ОПИСАНИЕ**

Сравнение двух значений по структуре. Recursive: примитивы – `===`, объекты – `keys + recurse`. Учитывает типы (Date, RegExp, Set, Map, циклические).

КАК РАБОТАЕТ

- 01 Если `===`, `true`.
- 02 `typeof` check.
- 03 `Array.isArray` vs `Object`.
- 04 Recurse по `keys`.
- 05 `WeakMap` для `cycles`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 State diff.
- 02 Test assertions.
- 03 Memoization.

ПОДВОДНЫЕ КАМНИ

- 01 `NaN === NaN` `false` – нужен `Object.is`.
- 02 Property order – не важен для `object`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `Object.is`?
- 02 Чем `shallow equal` от `deep` отличается?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#isEqual>

Реализуй flat(arr, depth = 1)

ОПИСАНИЕ

Уплощает вложенные массивы до depth уровня. Array.prototype.flat есть нативно.
Реализация: рекурсия + spread / concat.

КАК РАБОТАЕТ

- 01 Array.isArray check.
- 02 Recurse если depth > 0.
- 03 Concat results.
- 04 depth = Infinity для полного flatten.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Tree → list.
- 02 API responses normalisation.
- 03 Recursive structures.

ПОДВОДНЫЕ КАМНИ

- 01 Mutation вместо copy.
- 02 Большая depth – stack overflow на гигантских tree.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flatMap?
- 02 Чем concat от spread отличается performance-wise?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flat

ВОПРОС Q09

Реализуй unflatten({a.b.c: 1})

ОПИСАНИЕ

Объект с dot-нотацией → nested object. {a.b: 1} → {a: {b: 1}}. Полезно для form-state, query-params, config.

КАК РАБОТАЕТ

- 01 Split key по '.'.
- 02 Reduce: создавай вложенные objects.
- 03 Last segment – assign value.
- 04 Edge: массивы через [0] / arr[0].

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form serialization.
- 02 Config files.
- 03 Query params.

ПОДВОДНЫЕ КАМНИ

- 01 Path-injection – { 'a.b': 1, a: 2 } – collision.
- 02 Array indices vs keys.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое lodash.set?
- 02 Чем dot vs bracket нотация отличается?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#set>

PROMISE.ALL**POOL****RETRY****TIMEOUT****SEQUENTIAL**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PROMISE.ALL POOL RETRY TIMEOUT SEQUENTIAL

ВОПРОСЫ ДНЯ

- Q10 Реализуй `Promise.all(promises)`
- Q11 Реализуй `promisePool(tasks, concurrency)`
- Q12 Реализуй `retry(fn, retries, delay)`
- Q13 Реализуй `timeout(promise, ms)`
- Q14 Реализуй `sequentialPromises(tasks)`

ВОПРОС Q10

Реализуй Promise.all(promises)

ОПИСАНИЕ

Resolve когда все resolve. Reject если хоть один reject (fast-fail). Возвращает массив результатов в порядке входного массива.

КАК РАБОТАЕТ

- 01 Counter + results array.
- 02 Iterate, .then(resolve, reject).
- 03 Increment counter, results[i] = value.
- 04 Когда counter == length – resolve.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Параллельные independent fetches.
- 02 Multiple resources.

ПОДВОДНЫЕ КАМНИ

- 01 Один reject – все остальные продолжают, но результат уже rejected.
- 02 Без allSettled – теряем partial success.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Promise.allSettled отличается?
- 02 Что такое Promise.race?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all

Реализуй `promisePool(tasks, concurrency)`

ОПИСАНИЕ

Запускает `tasks` с ограничением `concurrency`. Когда один заканчивается – берёт следующий. Полезно для `rate-limiting`, `parallel uploads`, `batch processing`.

КАК РАБОТАЕТ

- 01 Очередь задач.
- 02 Worker-pool: запускай `concurrency` задач.
- 03 On task done – следующая.
- 04 Promise resolved когда все завершились.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Batch upload.
- 02 API rate limit.
- 03 Background jobs.

ПОДВОДНЫЕ КАМНИ

- 01 Без `error handling` – один `fail` и `pool stuck`.
- 02 Ordering не гарантировано.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `semaphore`?
- 02 Как реализовать с `p-limit`?

ПОЛНАЯ СТАТЬЯ

<https://github.com/sindresorhus/p-limit>

ВОПРОС Q12

Реализуй `retry(fn, retries, delay)`**ОПИСАНИЕ**

Повторяет `fn` до `retries` раз с задержкой `delay`. Exponential backoff для rate-limited API. Catch + recurse + setTimeout.

КАК РАБОТАЕТ

- 01 try await `fn()`.
- 02 catch – если `retries > 0`, recurse с `retries-1`.
- 03 setTimeout для backoff.
- 04 Exponential: `delay * 2^attempt`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Network requests.
- 02 Flaky API.
- 03 Rate-limited services.

ПОДВОДНЫЕ КАМНИ

- 01 Retry на permanent errors (404, 401) – бесполезно.
- 02 Без jitter – thundering herd.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое exponential backoff с jitter?
- 02 Чем circuit breaker отличается?

ПОЛНАЯ СТАТЬЯ

<https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>

Реализуй `timeout(promise, ms)`

ОПИСАНИЕ

Resolve если `promise` успел за `ms`, иначе `reject`. `Promise.race` между `promise` и `setTimeout`.

КАК РАБОТАЕТ

```
01 Promise.race([promise, timeoutPromise]).
02   timeoutPromise = new Promise((_, rej) => setTimeout(() => rej(), ms)).
03   clearTimeout если promise закончился раньше – leak prevention.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Slow API.
02 User-cancelable operations.
```

ПОДВОДНЫЕ КАМНИ

```
01 Без AbortController – request продолжает крутиться.
02 setTimeout не очищается – memory leak.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое AbortController?
02 Чем Promise.race от Promise.any отличается?
```

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/race

ВОПРОС Q14

Реализуй sequentialPromises(tasks)**ОПИСАНИЕ**

Запускает tasks последовательно: один-за-другим, ждёт окончания предыдущего. Reduce + .then.

КАК РАБОТАЕТ

```
01 tasks.reduce((p, task) => p.then(task), Promise.resolve()).
02 Каждый task – () => Promise.
03 Ошибка прерывает цепочку.
04 Aggregator: results[].
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Зависимые операции.
02 Сохранение порядка.
03 Database transactions.
```

ПОДВОДНЫЕ КАМНИ

```
01 В promiseAll вместо последовательности – race condition.
02 Reject стопит всё – нужна обработка.
```

МОГУТ ДОСПРОСИТЬ

```
01 Чем Promise.all отличается?
02 Что такое async iterator?
```

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

CHUNK**GROUP****UNIQ****INTERSECT****DIFF**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

CHUNK GROUP UNIQ INTERSECT DIFF

ВОПРОСЫ ДНЯ

Q15 Реализуй `chunk(arr, size)`Q16 Реализуй `groupBy(arr, key)`Q17 Реализуй `uniq(arr)` и `uniqBy(arr, key)`Q18 Реализуй `intersection(a, b)`Q19 Реализуй `difference(a, b)`

ВОПРОС Q15

Реализуй `chunk(arr, size)`

ОПИСАНИЕ

Разбивает массив на куски размера `size`. Полезно для batch-обработки, pagination, virtualization.

КАК РАБОТАЕТ

- 01 Loop с шагом `size`.
- 02 `slice(i, i + size)`.
- 03 Push в результат.
- 04 Edge: `size <= 0`, пустой `arr`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Batch operations.
- 02 Virtual scroll.
- 03 Pagination.

ПОДВОДНЫЕ КАМНИ

- 01 Mutation `arr`.
- 02 `size = 0` – infinite loop.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое generator-based `chunk`?
- 02 Чем `slice` от `splice` отличается?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#chunk>

Реализуй groupBy(arr, key)

ОПИСАНИЕ

Группирует массив объектов по key (или iteratee). Возвращает {key: [items]}.

КАК РАБОТАЕТ

- 01 reduce с accumulator.
- 02 Compute key (string или fn).
- 03 acc[key] = acc[key] || [].
- 04 push item.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Reports.
- 02 Filters by category.
- 03 Aggregations.

ПОДВОДНЫЕ КАМНИ

- 01 Mutation acc – но не arr.
- 02 key undefined – все попадают в 'undefined'.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое countBy?
- 02 Чем reduce от for-loop отличается perf?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#groupBy>

ВОПРОС Q17

Реализуй `uniq(arr)` и `uniqBy(arr, key)`

ОПИСАНИЕ

`uniq`: уникальные значения. `uniqBy`: уникальные по `key`. `Set` для `primitives`, `Map` для `objects`.

КАК РАБОТАЕТ

```
01  uniq: [...new Set(arr)].
02  uniqBy: Map с key → item.
03  Iterate, if (!map.has(key)) map.set.
04  Object.values(map).
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Dedup arrays.
02  Filter duplicates.
```

ПОДВОДНЫЕ КАМНИ

```
01  Set не работает для objects.
02  JSON.stringify для key – slow.
```

МОГУТ ДОСПРОСИТЬ

```
01  Что такое Set?
02  Чем Map от Object для cache отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#uniq>

Реализуй `intersection(a, b)`

ОПИСАНИЕ

Элементы которые есть в обоих массивах. Set + filter.

КАК РАБОТАЕТ

```
01 setB = new Set(b).
02 a.filter(x => setB.has(x)).
03 Для objects – кастомный equality.
04 Edge: пустые arr.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Common items.
02 Overlap detection.
```

ПОДВОДНЫЕ КАМНИ

```
01 O(n*m) с indexOf – slow.
02 Duplicates сохраняются если есть в a.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое Set.prototype.has complexity?
02 Чем difference от intersection отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#intersection>

ВОПРОС Q19

Реализуй difference(a, b)

ОПИСАНИЕ

Элементы из a которых нет в b.

КАК РАБОТАЕТ

- 01 setB = new Set(b).
- 02 a.filter(x => !setB.has(x)).
- 03 Для objects – custom comparator.
- 04 Edge: a пустой → [].

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Diff arrays.
- 02 Removed items.

ПОДВОДНЫЕ КАМНИ

- 01 Order имеет значение – $a-b \neq b-a$.
- 02 Set лучше для performance.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое symmetric difference?
- 02 Чем XOR от diff отличается?

ПОЛНАЯ СТАТЬЯ

<https://lodash.com/docs/#difference>

EVENT EMITTER**OBSERVABLE****ITERATOR****GENERATOR**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

EVENT EMITTER

OBSERVABLE

ITERATOR

GENERATOR

ВОПРОСЫ ДНЯ

Q20 Реализуй EventEmitter

Q21 Реализуй простой Observable

Q22 Реализуй Iterator-protocol

Q23 Реализуй generator function для range

ВОПРОС Q20

Реализуй EventEmitter

ОПИСАНИЕ

Pub/sub: on(event, handler), off, emit(event, ...args). Map<string, Set<handler>>.

КАК РАБОТАЕТ

```
01 events: Map<string, Set>.
02 on: events.set(event, set.add(handler)).
03 off: set.delete(handler).
04 emit: iterate set, вызывай handler(...args).
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Decoupled communication.
02 Plugin architecture.
```

ПОДВОДНЫЕ КАМНИ

```
01 Memory leak – handlers не отписаны.
02 emit во время handler iteration – copy set.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое once-handler?
02 Чем EventEmitter от RxJS отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://nodejs.org/api/events.html>

Реализуй простой Observable

ОПИСАНИЕ

subscribe(observer) → unsubscribe. Stream значений через next, error, complete.

КАК РАБОТАЕТ

```
01 new Observable(subscriber).
02 subscribe(observer): вызывает subscriber(observer).
03 return unsubscribe.
04 operators: pipe, map, filter.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Streams.
02 Reactive UI.
03 Event-driven.
```

ПОДВОДНЫЕ КАМНИ

```
01 Hot vs cold observable confusion.
02 Без teardown – memory leak.
```

МОГУТ ДОСПРОСИТЬ

```
01 Чем RxJS Observable от Promise отличается?
02 Что такое Subject?
```

ПОЛНАЯ СТАТЬЯ

<https://rxjs.dev/guide/observable>

ВОПРОС Q 2 2

Реализуй Iterator-protocol

ОПИСАНИЕ

Объект с `next()` возвращающий `{value, done}`. `for-of` поддержка через `[Symbol.iterator]`.

КАК РАБОТАЕТ

- 01 `[Symbol.iterator]()` return `this`.
- 02 `next()`: return `{value, done: false}` или `{value: undefined, done: true}`.
- 03 `for-of` использует протокол.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Кастомные коллекции.
- 02 Lazy sequences.

ПОДВОДНЫЕ КАМНИ

- 01 `return` перед концом – `for-of` не вызовет `next`.
- 02 `return` / `throw` – `cleanup`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `async iterator`?
- 02 Чем `generator` от `iterator` отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Iteration_protocols

Реализуй generator function для range

ОПИСАНИЕ

`function* range(start, end, step). yield` по одному значению. Lazy – не создаёт массив.

КАК РАБОТАЕТ

- 01 `function*` – generator.
- 02 `yield value`.
- 03 `for-of` или `[...range()]`.
- 04 Lazy: not eager evaluation.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lazy sequences.
- 02 Stream processing.

ПОДВОДНЫЕ КАМНИ

- 01 `[...gen()]` eager – теряет lazy преимущество.
- 02 Generator one-shot – нельзя `rewind`.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `async generator`?
- 02 Чем `yield*` отличается от `yield`?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/function*

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

4 ВОПРОСА

FETCH POLLING**STREAM****CACHE****LRU**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

FETCH POLLING

STREAM

CACHE

LRU

ВОПРОСЫ ДНЯ

Q24 Реализуй polling fetch с интервалом

Q25 Реализуй stream-обработку через ReadableStream

Q26 Реализуй простой in-memory cache с TTL

Q27 Реализуй LRU-cache

Реализуй polling fetch с интервалом

ОПИСАНИЕ

Регулярно дёргает API через interval. Stop условие: success / max attempts / external cancel.

КАК РАБОТАЕТ

- 01 setInterval / setTimeout chain.
- 02 await fetch.
- 03 If condition met → clear.
- 04 AbortController для cancel.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Job status.
- 02 Live data refresh.
- 03 Notifications.

ПОДВОДНЫЕ КАМНИ

- 01 setInterval не ждёт async – overlapping requests.
- 02 Без cancel – leak при unmount.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое long-polling?
- 02 Чем WebSocket лучше polling?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

ВОПРОС Q25

Реализуй stream-обработку через ReadableStream

ОПИСАНИЕ

Чтение по chunks из fetch response. Полезно для больших файлов, SSE, JSONL.

КАК РАБОТАЕТ

```
01 response.body.getReader().
02 while: const {done, value} = await reader.read().
03 Process chunk.
04 reader.releaseLock() в finally.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Large downloads.
02 SSE.
03 Streaming AI.
```

ПОДВОДНЫЕ КАМНИ

```
01 Без releaseLock – невозможно повторное чтение.
02 Не decode UTF-8 boundary – broken chars.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое TransformStream?
02 Чем SSE от WebSocket отличается?
```

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Streams_API

Реализуй простой in-memory cache с TTL

ОПИСАНИЕ

set(key, value, ttl). get(key) → undefined если expired. Map + timestamps + setTimeout cleanup.

КАК РАБОТАЕТ

- 01 Map<key, {value, expires}>.
- 02 set: expires = Date.now() + ttl.
- 03 get: if expires < now – delete + return undefined.
- 04 Optional: setTimeout для proactive cleanup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 API response cache.
- 02 Computed values.

ПОДВОДНЫЕ КАМНИ

- 01 setTimeout per item – много timers.
- 02 Cache size unbounded – нужен LRU.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stale-while-revalidate?
- 02 Чем TTL от LRU отличается?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/stale-while-revalidate>

ВОПРОС Q 27

Реализуй LRU-cache

ОПИСАНИЕ

Cache с capacity. При переполнении – удалить least recently used. Map (insertion order) + delete + set для re-insert.

КАК РАБОТАЕТ

- 01 Map хранит порядок insertion.
- 02 get: delete + set (поднимаем в конец).
- 03 set: если size >= capacity – удалить first key.
- 04 O(1) get/set через Map.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 URL cache.
- 02 Image cache.
- 03 Memoization с bound.

ПОДВОДНЫЕ КАМНИ

- 01 Без re-insert – LRU не работает.
- 02 Object вместо Map – нет ordering.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое LFU?
- 02 Чем Map от Object для LRU отличается?

ПОЛНАЯ СТАТЬЯ

<https://leetcode.com/problems/lru-cache/>

SCHEDULER**PRIORITY QUEUE****SCHEDULE**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SCHEDULER

PRIORITY QUEUE

SCHEDULE

ВОПРОСЫ ДНЯ

Q28 Реализуй scheduler с queue

Q29 Реализуй priority-queue

Q30 Реализуй scheduleAt(date, fn)

ВОПРОС Q28

Реализуй scheduler с queue

ОПИСАНИЕ

Tasks с приоритетом / time. Process по одному в idle. Можно через queueMicrotask, setTimeout, requestIdleCallback.

КАК РАБОТАЕТ

- 01 Queue.
- 02 schedule(task) → push.
- 03 Process в idle через requestIdleCallback.
- 04 Yield если deadline.timeRemaining() < 1.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Background work.
- 02 Animation pacing.
- 03 Interactive scheduling.

ПОДВОДНЫЕ КАМНИ

- 01 requestIdleCallback не везде поддержан.
- 02 Без yield – block UI.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое React scheduler?
- 02 Чем microtask от macrotask отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/Window/requestIdleCallback>

Реализуй priority-queue

ОПИСАНИЕ

Insert(value, priority). pop() возвращает highest-priority. Heap (binary) для $O(\log n)$.

КАК РАБОТАЕТ

- 01 Array-based heap.
- 02 Insert: push + bubble-up.
- 03 Pop: swap root with last, pop last, bubble-down.
- 04 Comparator для priority.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Task scheduling.
- 02 Dijkstra.
- 03 K-largest.

ПОДВОДНЫЕ КАМНИ

- 01 Linear search – $O(n)$.
- 02 Без bubble-up – broken heap invariant.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Fibonacci-heap?
- 02 Чем min-heap от max-heap отличается?

ПОЛНАЯ СТАТЬЯ

https://en.wikipedia.org/wiki/Binary_heap

ВОПРОС Q30

Реализуй `scheduleAt(date, fn)`**ОПИСАНИЕ**

Запускает `fn` в `date`. Через `setTimeout` с `delay = date - Date.now()`. Edge: max delay 32-bit (24.8 day).

КАК РАБОТАЕТ

- 01 `delay = date - Date.now()`.
- 02 `setTimeout(fn, delay)`.
- 03 Если `delay > MAX_INT` – chunked.
- 04 Возвращай cancel-функцию.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Notifications.
- 02 Reminders.
- 03 Cron-like.

ПОДВОДНЫЕ КАМНИ

- 01 Tab inactive – `setTimeout` throttled.
- 02 `Date.now` drift – long-term inaccuracy.

МОГУТ ДОСПРОСИТЬ

- 01 Чем cron от `setTimeout` отличается?
- 02 Что такое worker timer?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/setTimeout>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Реализовал debounce / throttle с leading + trailing
- ☐ Знаешь deep clone (structuredClone и ручную рекурсию)
- ☐ Реализовал Promise.all + pool + retry + timeout
- ☐ Знаешь chunk / groupBy / uniq / intersection / difference
- ☐ Реализовал EventEmitter + Observable
- ☐ Знаешь Iterator + Generator protocol
- ☐ Реализовал LRU + scheduler + priority-queue
- ☐ Можешь объяснить trade-offs каждого подхода

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru